

Future Interns – Cyber Security Task 1

Vulnerability Assessment Report

Prepared By:
Prarthana A

Target Website:
<https://demo.testfire.net>

Objective

The objective of this assessment is to identify publicly visible security weaknesses in the target application using passive security testing techniques without exploiting vulnerabilities..

Tools Used

TOOL	PURPOSE
Nmap	Port and service scanning
OWASP ZAP	Vulnerability assessment
Browser Developer Tools	Header inspection
Canva	Report design and documentation

Nmap Scan Result

```
C:\Windows\system32\cmd.e: x + v
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ASUS>nmap -Pn -sV testphp.vulnweb.com
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-12 21:13 +0530
Nmap scan report for testphp.vulnweb.com (44.228.249.3)
Host is up (0.0055s latency).
rDNS record for 44.228.249.3: ec2-44-228-249-3.us-west-2.compute.amazonaws.com
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
53/tcp    open  domain  Mikrotik dnssd

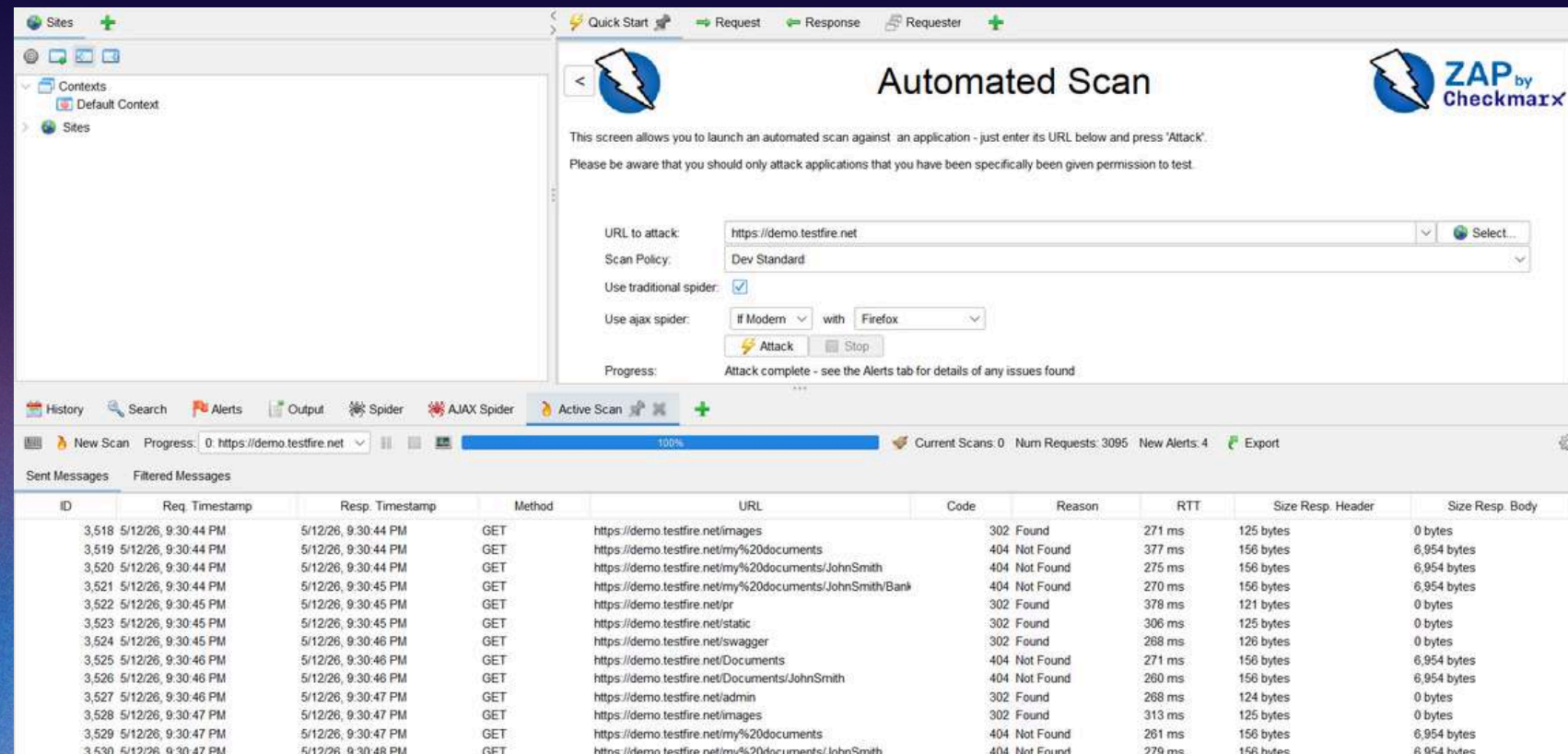
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 18.12 seconds

C:\Users\ASUS>
```

An open DNS-related service was detected during the scan. Exposed services may provide useful information to attackers if not properly secured. Recommendation: Restrict unnecessary exposed services using proper firewall rules.

Risk Level: Low

OWASP ZAP Findings



Automated Scan

This screen allows you to launch an automated scan against an application - just enter its URL below and press 'Attack'. Please be aware that you should only attack applications that you have been specifically given permission to test.

URL to attack:

Scan Policy:

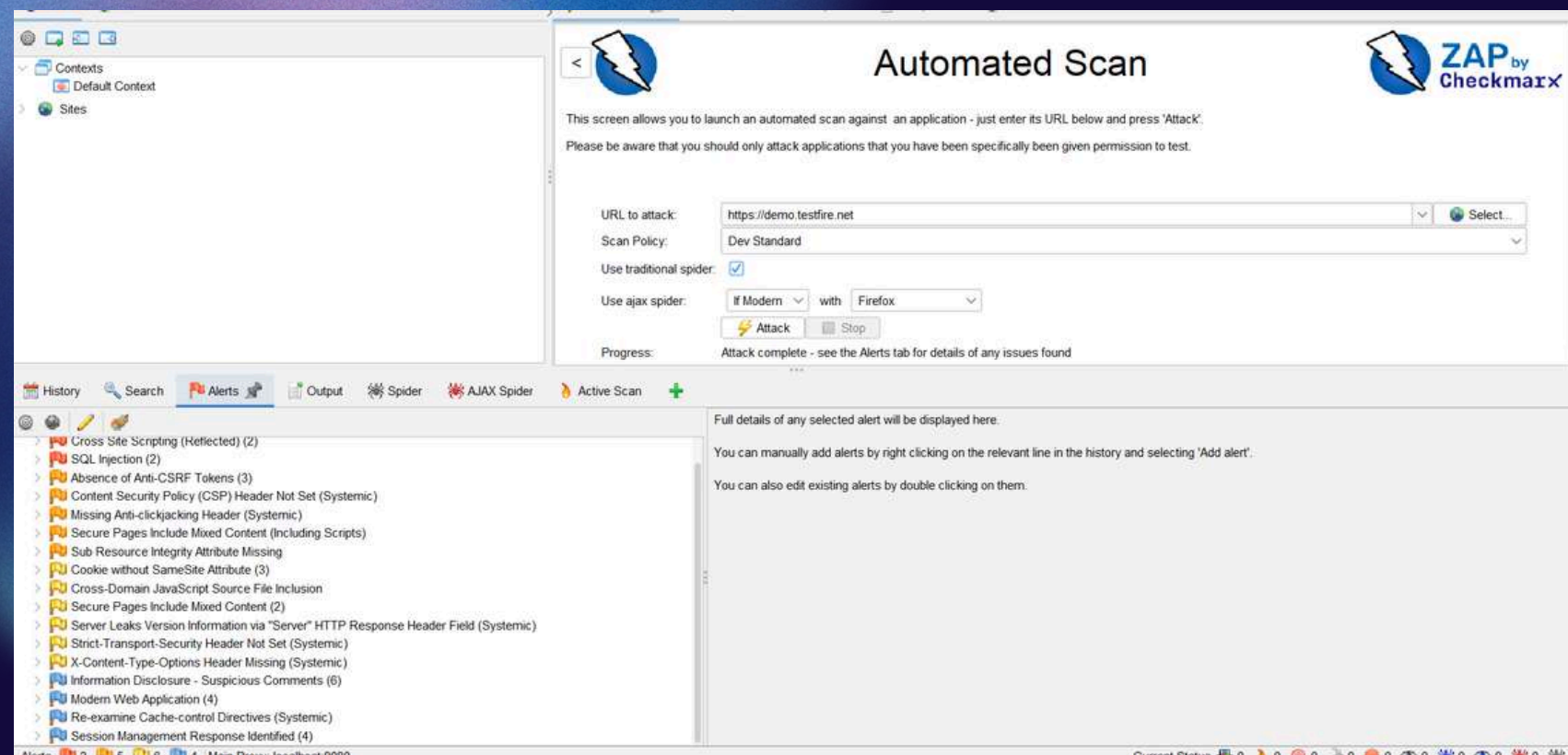
Use traditional spider: ☒

Use ajax spider: with

Attack

Progress: Attack complete - see the Alerts tab for details of any issues found

ID	Req. Timestamp	Resp. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Header	Size Resp. Body
3,518	5/12/26, 9:30:44 PM	5/12/26, 9:30:44 PM	GET	https://demo.testfire.net/images	302	Found	271 ms	125 bytes	0 bytes
3,519	5/12/26, 9:30:44 PM	5/12/26, 9:30:44 PM	GET	https://demo.testfire.net/my%20documents	404	Not Found	377 ms	156 bytes	6,954 bytes
3,520	5/12/26, 9:30:44 PM	5/12/26, 9:30:44 PM	GET	https://demo.testfire.net/my%20documents/JohnSmith	404	Not Found	275 ms	156 bytes	6,954 bytes
3,521	5/12/26, 9:30:44 PM	5/12/26, 9:30:45 PM	GET	https://demo.testfire.net/my%20documents/JohnSmith/Bank	404	Not Found	270 ms	156 bytes	6,954 bytes
3,522	5/12/26, 9:30:45 PM	5/12/26, 9:30:45 PM	GET	https://demo.testfire.net/pr	302	Found	378 ms	121 bytes	0 bytes
3,523	5/12/26, 9:30:45 PM	5/12/26, 9:30:45 PM	GET	https://demo.testfire.net/static	302	Found	306 ms	125 bytes	0 bytes
3,524	5/12/26, 9:30:45 PM	5/12/26, 9:30:46 PM	GET	https://demo.testfire.net/swagger	302	Found	288 ms	126 bytes	0 bytes
3,525	5/12/26, 9:30:46 PM	5/12/26, 9:30:46 PM	GET	https://demo.testfire.net/Documents	404	Not Found	271 ms	156 bytes	6,954 bytes
3,526	5/12/26, 9:30:46 PM	5/12/26, 9:30:46 PM	GET	https://demo.testfire.net/Documents/JohnSmith	404	Not Found	260 ms	156 bytes	6,954 bytes
3,527	5/12/26, 9:30:46 PM	5/12/26, 9:30:47 PM	GET	https://demo.testfire.net/admin	302	Found	268 ms	124 bytes	0 bytes
3,528	5/12/26, 9:30:47 PM	5/12/26, 9:30:47 PM	GET	https://demo.testfire.net/images	302	Found	313 ms	125 bytes	0 bytes
3,529	5/12/26, 9:30:47 PM	5/12/26, 9:30:47 PM	GET	https://demo.testfire.net/my%20documents	404	Not Found	261 ms	156 bytes	6,954 bytes
3,530	5/12/26, 9:30:47 PM	5/12/26, 9:30:48 PM	GET	https://demo.testfire.net/my%20documents/JohnSmith	404	Not Found	279 ms	156 bytes	6,954 bytes



Automated Scan

This screen allows you to launch an automated scan against an application - just enter its URL below and press 'Attack'. Please be aware that you should only attack applications that you have been specifically given permission to test.

URL to attack:

Scan Policy:

Use traditional spider: ☒

Use ajax spider: with

Attack

Progress: Attack complete - see the Alerts tab for details of any issues found

Full details of any selected alert will be displayed here.

You can manually add alerts by right clicking on the relevant line in the history and selecting 'Add alert'.

You can also edit existing alerts by double clicking on them.

- Cross Site Scripting (Reflected) (2)
- SQL Injection (2)
- Absence of Anti-CSRF Tokens (3)
- Content Security Policy (CSP) Header Not Set (Systemic)
- Missing Anti-clickjacking Header (Systemic)
- Secure Pages Include Mixed Content (Including Scripts)
- Sub Resource Integrity Attribute Missing
- Cookie without SameSite Attribute (3)
- Cross-Domain JavaScript Source File Inclusion
- Secure Pages Include Mixed Content (2)
- Server Leaks Version Information via "Server" HTTP Response Header Field (Systemic)
- Strict-Transport-Security Header Not Set (Systemic)
- X-Content-Type-Options Header Missing (Systemic)
- Information Disclosure - Suspicious Comments (6)
- Modern Web Application (4)
- Re-examine Cache-control Directives (Systemic)
- Session Management Response Identified (4)

Vulnerability	Risk
Cross Site Scripting (XSS)	High
SQL Injection	High
Missing CSP Header	Medium
Missing Anti-Clickjacking Header	Medium
Cookie Without SameSite	Medium
Server Information Disclosure	Low

Detailed Findings

FINDING 1 Cross Site Scripting (XSS)

RISK	High
EXPLANATION	Attackers may inject malicious scripts into webpages viewed by users.
RECOMMENDATION	Validate and sanitize all user inputs properly.

FINDING 2 SQL Injection

RISK	High
EXPLANATION	Improper input validation may allow attackers to interact with the backend database.
RECOMMENDATION	Use parameterized queries and proper input validation techniques.

FINDING 3 Missing CSP Header

RISK	Medium
EXPLANATION	The absence of a Content Security Policy increases the risk of malicious script execution.
RECOMMENDATION	Implement appropriate Content Security Policy headers.

Conclusion

The vulnerability assessment identified several security weaknesses within the target application, including high-risk vulnerabilities such as SQL Injection and Cross-Site Scripting. Implementing secure coding practices, secure headers, and proper input validation can significantly improve the overall security posture of the application.